

ABRA — Technical Documentation

Version 5.205.0 · Last updated 2026-08-28

5.205.0 — END THE SPRINT. DELETE THE LOG. READ THE GATE, DO NOT REMEMBER IT.

The MEDICHAM sprint started on 2026-08-10. The owner stopped it on 2026-08-28. During the sprint, each change wrote one row to a running log. The full document pass was deferred. The log had 274 rows. The changelog has 233 releases for the same period. The reports folder has 189 accounts. This pass writes up the sprint. Then it deletes the log. The deletion re-arms the full document rule. Do not read a number from the log. The log is deleted. Read a number from an artifact.

THE MEASUREMENT TOOL WAS THE DEFECT SIX TIMES OR MORE. Examine the instrument before you change the engine. This is the order of work. One night the tool made 30 accusations. 23 accusations were an incorrect test. 7 accusations were an incorrect rule. 1 accusation was a true engine defect. Do not repair 30 items. Repair the tool first. A red demonstration that is not written to its artifact cannot fail a gate. Write the demonstration to the artifact. An expired certificate is not a defect. Give each certificate a date. Refuse an expired certificate.

PUT A FINALISING MIX AT THE END OF A HASH THAT ADDRESSES A DIE. The functions `midEventHash` and `midHash` ended with `h = Math.imul(h ^ c, 0x01000193)`. There was no operation after the last round. FNV-1a has no diffusion after its last round. The last field of a draw address is the arrival index `nth`. The index has one digit for a usual move. A one-digit change is a change of four low bits. The hash value moves. The hash value does not re-draw. The maximum circular shift was 0.0351571. The correct value is near 0.5. Two arrivals in sequence shared a damage bucket 89.5% of the time. The correct value is 6.25%. The lag-1 autocorrelation was 0.8873. The correct value is near 0. The marginal hit rate was 0.9214. The target was 0.9. **The marginal rate was always correct.** Do not test a die on its marginal rate only. Test the die one step at a time. Add `fmix32` after the last round. The whole-game difference count went from 3 games to 14 games. This is a repair of the instrument. This is not a new defect.

A FIGURE FROM BEFORE 2026-08-27 IS VOID. IT IS NOT STALE. A void figure is not evidence. The comparison behind it covered a small part of the outcome space. Do not cite a void figure. Do not put a warning beside a void figure. Withhold the figure. Old paragraphs stay in this document. They are a dated record. They are not a current result. Run `node engine/status.js` for a current figure. An empty field means that no artifact knows the answer.

THE GATE IS SHUT. FIVE CLAUSES OF EIGHT GIVE NO ANSWER. Read `data/quarantine-stamp.json`. The field `gate_open` is false. The five clauses are the three roster stages, the whole-game differential, and the staged mechanics comparison. Each of the five artifacts records engine release `5f3f7141227c`. The tree has a different release now. An artifact that records a different release describes a different program. Withhold every count in it. Do not print a count with a caution beside it. A reader takes the count and leaves the caution. Run the measurement again. This is the only remedy.

A CHECKOUT CAN CHANGE A RELEASE IDENTIFIER WITHOUT A CHANGE TO THE ENGINE. One source of 26 moved. The source is `data/tags.json`. The copy in the release and the copy in the tree are equal after newline normalisation. Both parse to the same object. The content is the same. The digest is not the same. This repository sets `core.autocrlf` to true. A generator writes LF. A checkout writes CRLF. Cut the release over the bytes that a checkout gives. Do not cut over the bytes that a generator gives. This event is recorded for 2026-08-26 in `docs/ENGINE.md`. It has occurred two times.

THE CENSUS IS COMPLETE. THE CENSUS IS A LABORATORY. Read `data/mechanics-census.json`. There are 780 probed, 780 live and 0 missing. There are 780 armed and 0 unarmed. The census stages one scenario for each mechanic. Usage has no effect on the census. The census answers the question "is this correct". It does not answer the question "does this matter". Use the pinned team pool for the second question. The two instruments are not equivalent.

THE DAMAGE COMPARISON AGREES. THE SCOPE IS SMALL. STATE THE SCOPE. Read `data/engine-diff.json`. There are 6000 compared, 6000 agreed and 0 disagreed. The result is 0 disagreements at the midpoint. The result is 0 disagreements at each of the 16 band indices. The field `scope` says damage only. There are no items and no abilities in this comparison. Turn order, status duration and switching are not in this comparison. The field `skipped_multihit` is 134. The field `skipped_ability_multihit` is 17. The tool calls the single-hit entry point. The tool does not call the volley loop. **The tool has never applied a multi-hit move.** Do not quote this result as a general agreement.

"THE BOARDS AGREE" MEANS 33 LEAVES OF 80. Run `tests/probe_uncompared_leaves.js`. It reads 500 moves, 201 abilities and 148 items. The mechanics write 80 different leaves. The comparison reads 33 leaves. The comparison declares 4 leaves. 43 leaves are in no list. 25 of the 43 can be on the board at the turn boundary. A leaf that is not read looks the same as a leaf that agrees. Add a leaf to one list or the other.

THE QUARANTINE IS NOT LIFTED. TWO FACTS ARE TRUE AT THE SAME TIME. The computed condition is not met. The gate is shut. Every artifact below the simulator stays withheld. The owner set a narrower bar on 2026-08-22. The bar is zero board-material differences and a clean roster. The last measurements met the narrower bar. Nobody has changed the gate to test the narrower bar. Do not resolve this in a document. Change the gate, or leave the gate shut.

A WITHHELD FIGURE BECOMES RE-RUNNABLE. IT DOES NOT BECOME TRUE. This applies to leaf calibration, to the rollout rungs, to the exploitability results and to the weights. The weight fit is owed. The engine controls it. Compute does not control it.

3.98.0 — READ THE TAG PARAMETER. DO NOT MATCH THE MOVE NAME. The moves Quick Guard and Wide Guard have the same four tags. The tag `oneTurnGuard` has a parameter `blocks`. The value is "priority moves" or "spread moves". The simulator did not read the parameter. It matched the name "wideguard" in three places. The move Quick Guard became the action `pass`. The action `pass` does nothing. This is 927 uses. Read the parameter. Store the guard by its move identifier. Derive the class at each read. Test the final priority. A status move with the ability Prankster has a priority of +1. Do not block a move without the flag `protect`. The move Feint has no such flag. Put the test beside the ability test, above the action dispatch. A status move does not reach the attack branch. Do not put the spread test there. A spread move needs one message for each protected body.

3.97.0 — GIVE EACH HIT ITS OWN ROLL. DO NOT MULTIPLY ONE ROLL. The function `dmgRange` made one damage roll. It multiplied the roll by the number of hits. This is correct only if every hit is the same. Four moves in this format have hits that are not the same. The move Triple Axel has a base power of `20 * move.hit`. The powers are 20, 40 and 60. The move Dragon Darts has the field `smartTarget`. Hit 1 goes to the target. Hit 2 goes to its partner. The move Beat Up has one hit for each usable team member. Each hit has its own base power. The move Fickle Beam has a 30% chance of double power. It is 80 or 160. It is never 104. Use a loop over the hits. Enter the loop only if the base power depends on the hit number. Do not use a mean for a chance. Draw the chance in the battle loop. Keep the mean only for a price.

3.96.0 — DERIVE TAG MEMBERSHIP FROM THE HANDLER. DO NOT MATCH A NAME. The rule `speedMult` matched the name "choicescarf". The item Iron Ball has the same handler `onModifySpe`. It did not get the tag. The consumer was correct. The producer was not. The rule `statMult` matched four names. All four names are `isNonstandard: 'Past'`. The rule could not give a tag to any item. No code read the tag. Read

`chainModify` from the handler. Read the species lock from the handler. Do not write a name. The item Oran Berry heals a flat amount. The handler is `heal(10)`. There is no `maxhp` in it. Use the field `restoresFlat` for a flat amount. Do not multiply a flat amount by the maximum HP.

3.95.0 — THE DAMAGE FUNCTION MUST HONOUR AN INTACT DISGUISE. The ability Disguise blocks the damage of a move. The move does 0 damage. The ability then does damage equal to `maxhp/8`. These are two different sources. Showdown reports them separately. The battle loop applied the ability damage. The function `dmgRange` did not apply the block. It gave the same result with the ability and without it. The function `formeOnHitAbsorbs` holds this fact. The function `dmgRange` calls it and returns 0. The battle loop calls it and applies the chip. Do not write this rule in two places. The guard in the loop must not read the damage. The damage is 0 when the block applies. Read the base power of the move and the type effectiveness instead.

3.94.0 — READ THE USER'S OWN BOOST FROM TWO FIELDS. A move can change the stats of its user. Showdown puts this fact in the field `self.boosts`. Showdown also puts this fact in the field `selfBoost.boosts`. The two fields are not the same. The field `self` applies on use. The field `selfBoost` applies only after the move hits a target. The file `build/build_engine_data.js` read `self.boosts` only. Two moves use `selfBoost.boosts`. These moves are Clanging Scales and Scale Shot. Their rows had no self-data. The function `selfBoostsOf` reads both fields. It prefers `self`. It writes a warning if a move has both fields. Do not merge the two fields without a warning.

3.93.0 — THE PARTIAL TRAP COUNTER STARTS AT THE DURATION OF THE CONDITION. The condition `partiallytrapped` has a duration. The duration is 5. The engine decrements the duration in the Residual event. The Residual event of the turn the trap lands also decrements it. The tag `partialTrap` had the field `turns` with the value `'4-5'`. This is the number of turns of chip damage. It is not the duration. Do not use `turns` for the counter. The tag now has the field `duration`. Read the counter from `duration`. The field `turns` stays. It answers a different question. No code reads it. The function `partialTrapShape` in `engine/tag_dex.js` derives every value from the condition. It reads `duration`, the range in `durationCallback`, the item in the callback, and the divisor in `onStart`. It returns null if it cannot read them. A null makes the tag absent and the family refuses.

3.92.0 — FIVE TEST FILES USED MOVES THAT ARE NOT IN THIS FORMAT. A move can have the property `isNonstandard` with the value `Past`. Such a move is not in this format. The property `exists` is still true for such a move. Do not use `exists` to ask if a move is in this format. Ask for `isNonstandard`. Three files used the move `Tackle` for a slot that does not act. This is not correct. It has no effect. The file `tests/test-priority-block.js` used the move `Splash` to make a slot do nothing. The engine has no row for `Splash`. The slot did nothing because the move was unknown. Use `CS.INERT_MOVE`. The file `tests/test-dead-volatile.js` used `exists` as its guard. The guard was always true. The file now selects its move from the format by property.

3.91.0 — THE PROBE HARNESS VALIDATES A STAGED BODY. IT DID NOT VALIDATE ONE BEFORE. The class `Battle` does not validate a team. A probe can give a Pokemon a banned item. The simulator accepts it. The engines then agree about a mechanic the format does not contain. A new function `checkLegal` is in `engine/champions_sim.js`. It takes a species, an ability, an item and a list of moves. It gives the set a legal stat spread and five validated filler Pokemon. It calls the `TeamValidator` of Showdown. It returns the problems of the subject only. The function divides the problems into two lists. The list `banned` holds problems of existence. The list `pairing` holds problems of compatibility. The file `tests/probe_pair.js` calls `checkLegal` before it builds a body. A problem in `banned` stops the probe always. A problem in `pairing` stops the probe unless the caller sets the flag `iKnowThisPairingIsIllegal`. The flag does not apply to `banned`.

The quiet control ability is exempt from `pairing`. The control must not change with the species. A new function `firstLegalMove` gives a move the species can learn. Use it for a slot that does not act. Do not write the name of a move for such a slot. The move `Tackle` is not in this format.

3.90.0 — THE ENGINE DRAWS A MULTI-HIT COUNT. IT USED AN EXPECTATION BEFORE.

The function `expectedHitsOf` returns the mean of a hit distribution. For the 2-5 family that mean is 3.1. This is correct for a price. It is not correct for a turn. The authority draws a count from a twenty-element table and the count is 2, 3, 4 or 5. A new function `rollHitsOf` draws the count. It takes a move identifier and a random-number function. It reads the range from the tag. For the range [2,5] it indexes the authority's table. For any other range it uses a uniform draw and increments `MEDFAILS.multiHitRangeNot2To5`. That counter reads 0. The battle loop calls `rollHitsOf` once for each use of a move. The call is made when the first target is priced. It is not made when the move hits nothing. The count travels to `dmgRange` on the seventh argument. `dmgRange` uses the expectation when no caller supplies a count. The effects step reads the same count. It rounded the expectation before. Do not compute a fact twice.

3.89.0 — THE ENGINE NOW READS THE CONDITION ON `buffsHolderOnHit`, AND FOUR HEALING MOVES WORK. The function `condHolds` evaluates a tag condition. Before this change it accepted two arguments: the condition and the body that carries the ability. That is sufficient for a condition about the body. It is not sufficient for a condition about the incoming move. The function now accepts a third argument. The third argument holds the hit: the critical-hit flag, the resolved move type, the move category and the move identifier. Four condition shapes are readable. A condition that is not readable is refused and counted in `MEDFAILS.buffOnHitUnknownCond`. That counter reads 0. The function `healParam` returns the size of a heal. It could only read a fraction stored as an array. The tag for Synthesis, Moonlight, Morning Sun and Strength Sap stores `heal: true`. The function therefore returned nothing and the move became a wasted turn. `healParam` now returns a recipe. The resolution site spends the recipe, because the weather can change between the click and the move. The heal uses `md4096`, which is the authority's own `modify` function. Do not use a plain fraction.

3.88.0 — TWELVE MOVES WERE PRICED OFF GENERIC GEN-9 DATA INSTEAD OF THIS FORMAT'S, AND THE BUILDER THAT FIXED THEM WAS ONE RUN AWAY FROM DELETING TEN SPECIES. Trop Kick read 70 where the format says 85, Mountain Gale 100 against 120 — ours low in all twelve, and MAG's own table had the right numbers the whole time, so the two engines disagreed on every one. Asking what a regeneration WOULD do, before running one, turned up 788 destructive changes waiting in the same builder and a header stamp whose regex had never once matched. `buffsHolderOnHit` also gained its condition by derivation — Anger Point only on a critical hit, Justified only on Dark — but **the engine does not read it yet and nothing behaves differently**, which is said here rather than left to look like a fix.

3.87.0 — THE SIMULATOR USED TWO DIFFERENT WEATHER VALUES FOR ONE MOVE. The function `effMoveType` gives the type of a move. The battle loop uses it. It read the weather directly from the field. The function `dmgRange` gives the damage. It reads the weather from `effWeatherOf`. That function applies a PRIVATE weather, which one ability gives only to its own body. The two functions then disagree. A Weather Ball from that body had a damage value of 128-151 as a Fire move, and the battle loop refused it as a Normal move. Against a Ghost type, the damage was 0. CORRECTION: `effMoveType` now calls `effWeatherOf`. Do not copy the logic. One fact has one reader. RESULT: the census is 326 live of 326 probed, with 0 missing. The roster did not change. The 150-row damage comparison did not change, at 1 disagreement.

3.86.0 — EVERY PUBLISHED ARTIFACT HAS A WRITER NOW, INCLUDING THE ARM MILTANK ACTUALLY RUNS. The tool that answers "is this number still true" could only find a writer by an artifact's literal name beside a write call in two directories — so the mechanics census, the game differential, the interaction matrix and the deliberate roster, which are the four clauses of the MEDICHAM

gate, had no row at all. Not ok, not unsafe: absent. The graph goes 115 to 160 artifacts and the unknown set 61 to 16, membership derived through four ranked arms that each record how they matched. UNSAFE rises 13 to 20 because seven artifacts that were always unsafe are now visible; none left the set.

3.85.0 — THE WHOLE SITE WITHHOLDS NOW, AND THE DEPLOYED COPY WAS MISSING THE FILE THAT MAKES IT WORK. Five pages LOADED a quarantined artifact as data instead of quoting its verdict, so the citation checker could not see them at all; seven of the Stadium's fifteen cabinets now go dark, each keeping its seat and its button and answering with the quarantine instead of a number. The file that drives all of it, `quarantine-data.js`, did not exist under `app/` — which is the copy a visitor loads — so every guard there took the healthy path. That is the same failure as the day before, one directory over.

3.83.0 — THE PINCH FAMILY FIRES, AND THE ENGINE'S REFUSAL WAS CORRECT THE WHOLE TIME. Four abilities — Blaze, Torrent, Overgrow, Swarm — carry 9,141 sheet uses and had never fired. The simulator refused them because their condition was recorded as the SENTENCE "only below 1/3 HP", and this project's rule is that a guessed threshold is worse than no wire. `engine/tag_dex.js` now derives the condition as a STRUCTURE out of the official engine's own handler, `{cond:'hpFraction', of:'self', cmp:'<=', num:1, den:3}`, and `condHolds` in `engine/medicham2-browser.js` evaluates it. The fraction is never collapsed to a float: the test is `hp × den ≤ maxhp × num` in integers, because `maxhp × (1/3)` is smaller than `maxhp / 3` and would refuse a body at exactly one third the boost it is owed. Anything the engine still cannot read refuses and is counted. New gate: `tests/test-pinch-family.js`, 61 rows against the official engine, red at 31 of 61 before the change.

3.82.0 — THE FIRST ENGINE FIX OF THE QUEUE LANDS: THE VOLATILE DURATION FAMILY, 9,092 USES, AND IT WAS THE PERISH SONG BUG A SECOND TIME. Showdown decrements a volatile's duration inside the Residual event, so one applied on turn N has already spent a turn by the end of it. That defect was documented in this engine for Perish Song, fixed for Perish Song, and left standing for every other duration-bearing volatile. Taunt and Disable now match the official engine; Encore's counter row is gone and only a separate HP row remains. Whole-game board agreement rose 76.9% to 78.9% on a paired differential, the roster's moves queue fell from 52 to 50, and the census did not move. The previously published baseline could not be reproduced because the census digest and the team store had both shifted underneath it — so the run was discarded and re-taken paired. The delta is the measurement.

3.81.0 — THE QUARANTINE REACHED THE BOARD, AND THE CHECKER THAT POLICES IT WAS BLIND TO THE PUBLISHED COPY. Thirteen slots on ABRA WORLD's status board now render as a redaction bar rather than a number, each carrying the artifact, the reason and the command that re-runs it. Two defects in the guard itself were found doing it: its own selftest had gone red — an `all`-stage artifact was matching ANY requested stage name, so "a missing stage must FAIL" stopped being enforced — and its citation walker looked at `docs/` and `web/` but never at `app/`, which is the copy a visitor actually loads. Five withheld verdicts were being published from `app/` the whole time the check read green.

3.80.0 — THE DELIBERATE ROSTER'S INERT BUCKET COLLAPSED BY 94.1% OF ITS USAGE, AND A FAMILY OF ABILITIES WORTH 8,524 USES TURNS OUT NEVER TO HAVE FIRED. 124 abilities were falling through a catch-all that stages a plain attack, so the condition each one needs was never created and the roster honestly reported INERT — which reads as "nothing to test" when the truth is "never tested". Fifteen new shape rules take that bucket to 59 abilities / 4,261 uses, all 22 ability rules caught their own break, and nothing left in the bucket is above 500 uses. The first real defect it found: Blaze, Torrent, Overgrow and Swarm carry their below-1/3-HP condition as PROSE, and the consumer refuses any condition it cannot evaluate — so the pinch family has never once fired. The roster's artifacts are now written per stage, so the quarantine gate reads measurement rather than absence.

3.79.0 — EVERY FIGURE DOWNSTREAM OF MEDICHAM IS NOW WITHHELD RATHER THAN CAPTIONED, AND THE DELIBERATE ROSTER'S MOVES STAGE RAN FOR THE FIRST TIME. Will's standing call: *"all engines that take medicham's output should be regarded as out of date and we should stop referencing them until medicham is up to date and we can rerun them."* 34 of 114 artifacts are downstream and are no longer printed at all — R1, R2, R3, R4, leaf calibration and the weights among them — because a caption is not a quarantine: `PRE-CHANGE` had been printed beside those numbers for days and they went on being quoted anyway. Membership is derived from the dependency graph, not typed, and the gate that lifts it is computed from the differential and the roster, where a `MISSING` stage counts as failing. The roster gained 26 move shape rules and staged all 500 legal moves for the first time, returning a 79-row queue over ~15,000 uses. Its control arm was found to be measuring the `CONTROL` rather than the subject, which made six ability findings false; **Weather Ball, Sand Rush and Damp are retracted as defects and are correct.**

3.78.0 — THE SHEET'S REAL NATURE NOW REACHES BOTH ENGINES, AND THE TURN-1 NUMBER FELL. THAT IS THE INSTRUMENT GETTING HONEST. The whole-game differential built every body `Serious` while the stored sheet beside it said `Modest`, and with every body flat AND `Serious`, 326 of 357 species in the format share a Speed with some other species — so the rig `MANUFACTURED` speed ties and almost never tested a real speed differential. Carrying the declared nature cut the tied groups the resolver has to break from 348,595 to 243,467 over the same 1,998 games, a 30.2% fall. The instrument's own numbers went DOWN, as predicted before the run: the board at the end of turn 1 is identical in 97.4% of games flat against 97.3% natured, games whose board never parted 80.8% against 78.8%, and the median turn of first board divergence one turn earlier at 7. Encore divergences nearly doubled (10 to 19 games), which is what a duration volatile that only bites when turn order does looks like when turn order starts being tested. **THE SPREADS REMAIN ABSENT AND ALWAYS WILL BE** — a Showdown open team sheet does not reveal them (`"evs": null` on 173,784 of 173,784 stored bodies), so this narrows the declared gap and does not close it. Neither engine is told the other's answer: both are told the nature and each computes, and the alignment assertion still reads 0. It read 21 on the first run and all 21 were Ditto — entry-time Imposter had already transformed the medicham body, and the harness was writing the copied stat line onto Showdown's Ditto before the game began. Census 324 live, 0 missing, unchanged.

3.77.0 — CONFUSION DID NOT EXIST, AND BURN HAD NEVER BEEN ON A BOARD. The confusion volatile was written and never read or ticked, so Hurricane's secondary — 3,779 uses — fell through every branch, and the two berries that clear confusion looked dead because there was nothing to clear. The sleep counter was an ordering bug: the authority runs sleep before flinch, ours ran flinch first, so a body that was asleep AND flinched never ticked and woke a full turn late. Burn, by contrast, is **CORRECT** and was confirmed rather than changed — but it had never once been staged, because Will-O-Wisp is 85-accurate and the harness pin makes every sub-100 move miss. The freeze timer is correct too; what was missing was the instrument, which carried no freeze counter at all, so the engine's value could drift and no measurement would see it. Census 324 live, 0 missing.

3.77.0 — THE ACROSS-A-SWITCH ARM FOUND A DEFECT ONE DAY OLD THAT FIXING SOMETHING ELSE CREATED. A transform never reverts when the body leaves the field: the authority clears it in `clearVolatile`, and this engine sets the flag and never unsets it. Since the transform also overwrites the body's name, stats, types, moves, boosts and ability, a benched Ditto is **PERMANENTLY** the thing it copied — so the two engines then choose replacements from benches that no longer describe the same Pokemon, and worse, a Ditto can only ever transform **ONCE PER BATTLE**, because the guard refuses a second. Re-copying is the entire function of the Pokemon. Imposter first fired the day before, and the out-and-back scenario that exposes this only became expressible hours earlier. The roster's two owed arms — across a switch, and at the exact HP line — are both built, both red-demonstrated, and Speed Boost and Focus Sash both match.

3.77.0 — A STAGED SCENARIO CAN NOW SWITCH, SO A MID-TURN ENTRANT IS

EXPRESSIBLE FOR THE FIRST TIME. The scenario driver understood only a move; every other step became a pass, so no staged test could put a body on the field part-way through a turn. That single gap blocked three things at once: Speed Boost's entry gate, which exists only for a body that just switched in; Hunger Switch's flip and Zero to Hero's switch-out transform; and the whole across-a-switch arm of the roster. Four of the six engine defects found the day before were about a MOMENT rather than an effect, and no scenario without an entrant can express one. Verified end to end: Espathra switches in and reads +0 Speed in both engines on the turn it arrives, then +1 at the end of the next, with all 131 fields identical on both boundaries.

3.77.0 — ALL 316 ABILITIES STAGED DELIBERATELY, AND A FREE +6 ATTACK FELL OUT.

Anger Point and Justified are one defect twice: a conditional boost-on-being-hit whose condition is never checked, so Anger Point grants +6 Attack off an ordinary hit where it requires a crit, and Justified grants +1 off a Poison move where it requires Dark. Hustle applies no 1.5x Attack at all. Two facts about the instrument matter as much: Gastro Acid does not suppress an ability here, and since suppression is the ONLY control available to 23 abilities, checking that control against a known-live fixture is what stopped five more from being published as dead for the control's failure rather than their own. And a fact about the regulation rather than the simulator: 113 of 316 legal abilities have NO legal carrier, so the effective roster of this format is about 203.

3.77.0 — FIVE MECHANICS THAT DID NOTHING, AND ONE THAT WAS ALREADY RIGHT.

Imposter never transformed Ditto; Hunger Switch never flipped Morpeko; Knock Off took its 1.5x against an item it cannot remove; Fling never became an attack at all, because a base power of 0 made the click fail a `hasPower()` gate; and Roar's phaze branch held a Pokemon-first target, so a phaze after a pivot dragged nobody — the SIXTH site missed by the slot-first sweep, and at priority -6 the worst possible place to hold a body rather than a slot. Mawile's mega ability swap, which had been blamed for a whole family of Attack-stage divergences, WAS ALREADY CORRECT: the scenario was board-identical on its first run, and deleting the swap deliberately parts two fields at once, so the symptoms were real symptoms of a bug this engine does not have. Census 319/319 live, 0 missing; the staged harness now carries 24 scenarios, all clean and all breakable.

3.77.0 — THE DIFFERENTIAL IDENTIFIED A SWITCH TARGET BY TWO DIFFERENT KEYS.

The driver selects a bench member and records the species identifier. The official-engine side matched on that identifier. The MEDICHAM side matched on the display name. The two values are equal until a forme change alters the display name. Forme changes were added in the previous release. After a forme change, the values differ and the body cannot be selected. Neither side reported the failure: each returned "pass". One engine could therefore switch while the other did not. The differential now stamps one key at build time and both sides use it. A failed lookup is counted and printed. The count was 0 and 0 over 120 games.

WIRE 138-140 — THREE BOARD FAMILIES, AND A TARGETING MODEL THAT WAS WRONG WHENEVER ANYTHING MOVED (3.77.0).

Aimed at the three largest surviving board-divergence families of the 1,530-game run at release `288aee2e3501`. **Speed Boost fired a turn early:** Showdown gates it on `activeTurns`, which is 0 on the turn a body switches in, and this engine's own comment said the gate "is not expressible here" — true of `_turnsOut` and untrue since WIRE 135 added `_newlySwitched`, a reason that was correct when written and stale when read. **A move targets a SLOT, not a Pokemon** (Will: "*we gotta target slots, not mons*"): `Battle#getTarget` resolves from `targetLoc` at execution time, and five of this engine's seven branches held the object they aimed at, so Charm and Parting Shot (10,535 uses: Charm 1,625 + Parting Shot 8,910, `data/tags.json`; this read 7,184 when first written and the corpus has grown since) dropped stats on a body sitting on the BENCH. One shared reader now answers it everywhere, with `tracksTarget` (Snipe Shot, Stalwart) as the negative. **Ally Switch did not exist** — 202 uses resolving to a wasted turn — and it is the sharpest test of the slot rule, because both bodies stay on the field: before it, one

unimplemented move parted TEN board fields at the end of a single turn. Each was RED on a staged board before its wire and IDENTICAL after; mega evolution, checked in the same pass, was already correct. Census **311 → 313 live, 0 missing**; staged boards 18/18 identical and 18/18 breaks caught.

WIRE 133-137 — THE SPEED TIE, THE SWITCH-OUT CLASS, AND THE LAST MISSING

MECHANIC (3.74.o). The two engines have disagreed about every speed tie for the life of this project, and the cause is the SORT rather than the comparison. `Array.prototype.sort` is stable, so two equal actions keep their input order; the authority uses a selection sort whose swaps move UNTIED actions around, so the tied group is no longer in input order by the time the tie is resolved. No comparison rule can produce that from a stable sort. It is not confined to the test instrument: the same function orders every turn the bot plays, and 91.4% of legal species share a base Speed with another species. The engine now performs the same selection sort, and resolves the remaining tie with the random key it already drew — a fair coin under real dice, and the identity under a frozen die, so both engines choose the same body without either being told the answer. Taking "the later body" was refused: that is what the authority produces under the frozen die, not the rule of the game. Beside it, three defects that changed a board rather than a message: Zero to Hero changed forme on the RETURN instead of on the way out, Disguise never renamed the body, and a Pokemon that pivoted out paid its recoil, its drain and its Life Orb cost from the bench. Twelve mechanics that had never been probed are now probed; two of them were already correct and unproved. The mechanic census reads 310 live of 310 probed, and `missing` is zero for the first time.

THE TEST USED ONE SET OF DICE. IT NOW USES FOUR (3.73.o). The differential freezes each random value so that both engines get the same result. Before this release it froze each value one way only. The effect was that the speed tie always gave the same order, each move with accuracy below 100 always missed in both engines, and the damage roll was always the maximum. A move with accuracy below 100 had therefore never hit in this test. The test now runs four arms. Each arm freezes the values a different way. Each arm stays fully deterministic. The set of frozen values is recorded in the artifact. If two runs use different sets, the comparison is refused. This release also changes how coverage is counted. Before, a mechanic counted as covered when the engine clicked it. A click can do nothing: Haze removes stat changes, and there were no stat changes to remove. A mechanic now counts as covered only when the board changes as the tag says it must, or when a declared negative case is reached and it correctly does not fire. Five mechanics were counted as covered and did nothing. **Do not compare any result after this release with a result before it. Both changes alter which games the test plays.** One engine fault was found and is not yet corrected: when two Pokemon have equal Speed, the official engine moves the later one first and this engine moves the earlier one first.

ROADMAP #92 — THE DAMAGE-STAGE CLASS. FOURTEEN MULTIPLIERS WERE APPLIED AT THE WRONG STAGE AND FIVE WERE ABSENT (3.73.o). Showdown applies each multiplier at a STAGE — a base power, a stat, or the final damage — folds every handler at that stage into ONE modifier, and spends it ONCE. This engine applied about a third of them a stage late, and separately: Black Glasses on the final damage where the authority puts it on base power reads 109 against 108. That one-point shape is why it survived every existing check — both engines "apply Black Glasses", so the census saw it LIVE, the interaction matrix compares a ratio between arms, and the damage differential allows a 12% midpoint band by design. LANDED: the 18 type items, Muscle Band, Wise Glasses, Technician, Tough Claws, Sharpness, Iron Fist, Mega Launcher, Strong Jaw, Punk Rock, Sheer Force, Supreme Overlord, Expanding Force, Rising Voltage, Dry Skin and the -ate x1.2 into ONE base-power relay spent once; Thick Fat (73% wrong), Heatproof, Purifying Salt and Water Bubble (77%) into the STAT relay, because they modify a stat and not the damage; Helping Hand (wrong on 5 of 5 audited rows) and Friend Guard (21.4%) off the hit site and into the chains they belong in; Sniper out of the crit's plain multiply and into the final chain; and the rolled crit's POSITION into the damage range before the randomizer, where it was 46.5% wrong at the bottom roll and invisible at the top one every check pins. The four FIELD terrains were absent entirely — a Grassy-Terrain Earthquake was

priced at DOUBLE the real number. New gate `tests/test-damage-stages.js` is **1,728 of 1,728 exact** against the authority across all sixteen damage rolls and both crit states, and was shown RED on two deliberate reversions before being trusted. `damageBoost` is still NOT wired as a class and the reason is a property of the tag: it carries neither the stage nor the condition, so wiring it would hand Blaze a permanent x1.5 on 5,808 sheets. Census 293/294 → 298/299 live.

ROADMAP #81 WIRE 12 — FIVE ENGINE DEFECTS OFF THE TURN-1 BOARD, TWO OF THEM MIS-DIAGNOSED BEFORE THEY WERE FIXED (3.71.0). The auras (Fairy, Dark, Aura Break) are wired FIELD-WIDE at the base-power stage — they multiply one type for every body on the field, the foe's moves included, and Aura Break INVERTS to x0.75 rather than cancelling; exact against the official engine on 12 of 12 staged arms. Baton Pass and Shed Tail switch for the first time (`passesState` had been derived and never consumed, so Baton Pass was a no-op turn and Shed Tail paid half its user's HP to stand still). Curse is two moves and the engine had neither. Perish Song counted from 3 instead of 4 and therefore fainted every affected body on both sides a full turn early, on 1,141 corpus uses — the KO itself had always fired. And ROADMAP #81 WIRE 10's measured board regression is one line: the Life Orb toll was being paid by a move that MISSED. **Two of the five briefed diagnoses were wrong** — the tagger was not testing `selfSwitch === true`, and the substitute doll was not confounded, it was a regression this project introduced at WIRE 7 on a misquoted source line. Census 281/282 → 293/294 live.

THE INSTRUMENT WAS MEASURING ANNOUNCEMENTS, AND THE HEADLINE IS NOW THE BOARD AT THE END OF TURN 1 (3.70.0). `engine/board_state.js` reads HP, status with its counters, items, all seven stat stages, aliveness, every field condition WITH ITS CLOCK and the persistent volatiles out of BOTH engines' live bodies at every turn boundary, after the whole residual phase. Read every figure from `data/state-ladder.json`. **The board at the end of turn 1 is identical in 56.0% of games at the pre-WIRE-1 baseline (1119/1998) and 66.9% at the top rung (1337/1998)**, peaking at 69.3% at WIRE 9; whole-game board agreement went 6.4% → 15.6% against a protocol number that read 1.8% → 10.3%, so the wires were real and the protocol number overstated them. **WIRE 10 is a regression the protocol instrument scored as an improvement** — 47 fewer clean turn-1 boards, and diffed per field it is one field, end-of-turn-1 HP wrong in 427 → 473 games. **41.0% of games whose narration parted inside turn 1 reached an identical board anyway.** The comparator proves itself first: 7 representation mappings red-demonstrated in both directions and 25 planted state divergences, each of which must be caught at the planted boundary and localised to the planted field — 25/25 on all fourteen arms.

ENGINE CORRECTNESS DOES NOT CHANGE THE LEAF (3.69.0).

`engine/leaf_engine_contrast.js` measures the effect of engine correctness on prediction quality. It writes `data/leaf-engine-contrast.json`. Read all figures from that file.

Procedure. The tool scores the in-game leaf on 8,883 positions. It uses 200 rollouts for each position. It uses the same seed for a given position in both arms. It reads the engine from two frozen releases. The two releases differ in one file only: `engine/medicham2-browser.js`. The tool stops if they differ in more than that file.

Results.

- The paired Brier difference is 0.0000. The 95% confidence interval is [-0.0007, +0.0007].
- The split-half noise floor is 0.000642. The smallest detectable effect is 0.001013.
- The confidence interval is smaller than the smallest detectable effect. The result is a null result. The sample is large enough. Do not report the result as "not detected".
- The McNemar test gives 37 correct for the new engine and 36 correct for the old engine. The p-value is 0.91.

Divergence depth does not predict leaf error. For the new engine, the Spearman rho is +0.0010 for lines and -0.0000 for turns. The minimum detectable rho is 0.0298. For the old engine, both values are positive (+0.031 and +0.029). A positive value is the opposite of the hypothesis.

The depth instrument is reliable. A control arm reads the same positions with the driver order reversed. The two readings agree at rho 0.836. The null result is therefore a property of the world and not of the instrument.

Calibration is the failure. The ECE is 0.1514. The leaf predicts values from 0.06 to 0.94. The observed win rate moves only from 0.466 to 0.594. When the leaf predicts 94%, it wins 59%.

To run the tool:

```
SHOWDOWN_PATH=... node engine/leaf_engine_contrast.js --write
```

The tool can resume. Use `--work <dir>` to reuse a previous run's arms. The tool re-runs 24 positions of each reused arm. The values must be identical. If they are not identical, the tool stops.

`engine/leaf_scoring.js` holds the scoring definitions. To check it against the published artifact:

```
node engine/leaf_scoring.js --verify
```

It compares 749 values. All values must match.

THE RELEASE LADDER — SEVEN FIXES DID NOT MOVE THE MEDIAN TURN (3.68.0, re-run 2026-08-07). `engine/wire_ladder.js` plays every frozen release of the wire series through the differential. It uses one pinned census and one team pool, so all eleven arms compare with each other and not only with their neighbour. **Read every figure from** `data/wire-ladder.json` — the figures below moved when ROADMAP #81 WIRE 7 was added and the whole ladder was replayed, so any earlier quotation of them is retracted. On 1,995 games for each arm, the median game stops after **one completed turn at every rung**. That number does not change. 64 games of 1,995 agree completely, against 6 games at the baseline. The depth of the first disagreement does change: the mean goes from 14.8 to 27.8 protocol lines, the 90th percentile from 30 to 89, and the MEDIAN first-divergence line from 13 to 16 — the first rung in the series to move it. The baseline arm ran first and last, with nine arms between them, and gave the same result. Therefore the table shows the engine change and not the run.

THE DIFFERENTIAL HAS RUN, AND MEGAS ARE IN IT (3.62.2). `engine/game_differential.js` plays a real stored team through MEDICHAM and through the official Showdown engine, step for step, against a stamped frozen release. **Read every figure from** `data/game-differential.json`, **never from this sentence** — the first version of this paragraph quoted a run that a later one replaced within the day, which is the drift this whole document set keeps having to correct.

At the time of writing it reports every measured game diverging, with the median parting after a single completed turn. **Mega bodies are now tested** (ROADMAP #31): no stone is stripped from the measured arm, and every mega choice Showdown offered was taken by both engines.

Two limits travel with any rate this instrument prints and must never be separated from it. Nothing past the first turn is exercised, because a game stops at its first divergence. And both sides are built Serious / 0 EVs / 31 IVs so the two engines compute the same stat line before *and* after a forme change — **this tests RULES, not the spreads the ladder actually brings.**

Change record for 3.62.2. The headline metric changed. It was the win rate. It is now the exploitability. Read ADR-003 for the decision. Read `docs/POKER-TO-POKEMON.md` for the theory.

The reason is a measurement from other persons. VGC-Bench is the only published work in this format. Its authors trained a policy on more than 700,000 human battle logs. They then improved it with PPO, self-play, fictitious play and double oracle. The policy defeated a World Championships competitor in a mirror match. The authors then trained a best response against each of their own agents. Almost all of the agents were approximately 100% exploitable. Their expert tester wrote that strong human players adapt and defeat the agent after sufficient successive games.

A fixed policy is a map from a state to an action. In a game with hidden information, a best response can find the weak states of such a map. This project therefore tests one claim: an agent that computes a new answer each turn is more difficult to exploit than an agent that recalls a stored answer. **This claim is not proven. It is the experiment.**

The speed figure also changed. ADR-001 recorded 29 against 3,401 battles/sec/core. It gave a ratio of 117x. The measurement was made again on this machine. Both engines used the same four teams. The teams are derived from the store. Each run was 8 seconds with a 60-turn limit. MEDICHAM gives 13,041 turns/sec and 217 battles/sec. `champions_sim` gives 523 turns/sec and 28 battles/sec. The ratio is 24.9x. Use `turns/sec`. Do not use `battles/sec`: MEDICHAM ran to its 60-turn limit and Showdown ran with `choose('default')` to a natural end, so a battle is not the same quantity of work in the two engines. The old figures stay in ADR-001. The decision in ADR-001 stays correct. The stated reason for that decision changed: the engine is justified if, and only if, the search gives a measured gain.

The work is now in four phases. Complete MEDICHAM. Then run gate ROADMAP #62. If the gate passes, build the search and measure the exploitability against approximately 100%. If the gate does not pass, use the method of VGC-Bench: behaviour cloning, then PPO with self-play, fictitious play and double oracle. Phase 4 is a result. It is not a failure.

Change record for 3.47.0. Two artifacts were computed from an engine that then changed. Do not quote such an artifact. First measure whether the change moved the feature function: run every feature column through the old engine and the new engine on the same rows. All 58 columns were identical on 1,751,688 rows from 9,230 games. Then the two artifacts were computed again with the new engine. The result is the same: the model puts less probability on an action no human chose, **-0.002613, 95% CI [-0.003650, -0.001672]**, on 48,274 held-out decisions. `board.js` gives the priority rule a body with no type list. This is wrong for 0 of 1,751,688 rows. Do not change it yet.

Change record for 3.42.0. The fit checks whether a recorded action was a click. 1,336 of 241,927 actions were not clicks. Encore replaced 1,116 of them. A phazing move dragged in 220 of them. These actions are removed from the training labels. They are counted. 3,260 redirected attacks are kept. They are fitted over a set of two possible targets, not one certain target. The estimator was tested on data with known answers first. It recovers 97.4% of a planted error. The model now puts less probability on an action no human chose: **-0.002614, 95% CI [-0.003647, -0.001610]**. The redirection change did not improve anything that can be measured. The overall top-1 accuracy did not change. Two budget counters were replaced. The old counter measured two different things at once.

Change record for 3.45.0. The matrix did not test a move that can miss. It also did not test a move with a chance side effect. Many of these are the moves that players use most. The rule was wrong. Both engines use the same fixed die, so a miss happens in both engines and cancels. The removed counts are in `CHANGELOG.md`.

The two dice were not the same die. The harness set one die to the middle value. It set the other die to a different rule. Showdown checks accuracy with the second die. So every move below 100 accuracy missed in the official engine and hit in the simulator. This made false disagreements. It also made many cases look empty. Nobody saw this, because all of these moves were removed from the test first.

The harness now checks that the two dice agree. The check runs when the file loads. The matrix stages 2,300 pairs of 8,795. At this release 1,453 of them could occur, and the matrix agreed with the official engine on 1,436 of these. Release 3.46.0 changed both of those counts. Twelve new faults in the simulator are recorded. They are not repaired in this release.

Change record for 3.46.0. A test needs the other Pokémon to do something while it is hit. The harness gave it a move for this. If the Pokémon could not learn one of six safe moves, the harness gave it Protect. Protect stops the move under test. The test then showed no result. A test with no result looks the same as a test that cannot work. 379 of 2,300 tests were built this way.

The harness no longer uses Protect for this. It uses any move that the Pokémon aims at itself and that does not block. If no such move exists, the test is not built, and the reason is recorded. A check stops the run if a Pokémon is given a Protect for this purpose.

From `data/interaction-matrix.json` : 1,642 of the 2,250 tests can occur, and the matrix agrees with the official engine on 1,642 of these — every one, with `part` at 0. The artifact also records 12 more disagreements in buckets the gate discards. These are real. Read `off_gate` and `off_gate_rows` , not the agreement rate alone.

The control move for a test whose reactor is a MOVE must not carry the property under test. The harness used to read that property from a usage-ranked index, which omits a move that carries the property and that nobody plays. Fifteen contact moves are omitted in this way. One of them was selected as a control, so both arms of the test made contact and 57 tests showed no result. The property is now read from the move itself. `engine/linkage_carrier.js` holds the one rule, and both the index builder and the test generator call it.

Many tests that showed no result now show one. The count of these is in `CHANGELOG.md` . Three new faults in the simulator are recorded. One new fault in the harness is also recorded. The harness fault must not be counted against the simulator.

Change record for 3.44.0. Psychic Terrain stops a fast move only if the target stands on the ground. The simulator stopped the move against every target. A target that flies is not on the ground. A Flying type, a Pokémon with Levitate and a Pokémon with an Air Balloon all fly. A Pokémon that holds an Iron Ball is on the ground, even if it flies. The simulator now has one function for this question. Four parts of the simulator ask that function. Before this release three parts each had their own copy of the rule, and the copies did not agree. Grassy Terrain also used a copy. That copy healed a Pokémon with Levitate. Grassy Terrain must not heal a Pokémon that flies. All expected results come from the official engine. The count of tested mechanics is 210 of 213.

Change record for 3.43.0. The interaction matrix now checks its own arithmetic. The rule is `theoretical = staged + dropped` , for each axis. The generator stops if the rule is broken. The rule found three faults. The count of theoretical pairs was too small by 170. The depth limit on the type axis lost one pair each time it applied, 32 times. Four cases were counted in two result groups. After the repair the matrix stages 1,675 pairs of 8,676, and 1,031 of them can occur. The matrix agrees with the official engine on 1,027 of these 1,031 cases.

The agreement figure is lower than in 3.41.0. The simulator did not get worse. The earlier figure of 899 of 899 was measured on a smaller set. The four cases that disagree were not made before this release.

Change record for 3.41.0. The simulator covers 202 of 205 probed mechanics. Three are missing. Each has a written reason. The interaction matrix agrees with the official engine on 899 of 899 cases. The pair-layer fit now uses all four team-sheet channels (95,886 turns, 99.7% channel reach). The value of the sheet was measured on 44,982 held-out decisions: the likelihood gain is real; a top-1 gain is not shown. The pory family was refit on the current corpus. The DEAD-tag count fell from 61 to 38.

Change record for 3.40.0. The simulator covers 181 of 186 probed mechanics. Five are missing. Each has a written reason. The two rulebook files were compared. 151 facts were comparable. Two facts did not agree. The engine now reads the format's own secondary-effect chance. The MAG fit now uses all four team-sheet

channels. A counter proves the channels reached the board (99.67% of 231,722 decisions). The pair-layer fit does not use them yet. The coverage plan changed: mutation tests come before the handler registry. See `COVERAGE-PLAN-REVIEW.md` for the reasons. ABRA has no exploitability number.

Change record for 3.56.0. The simulator did not use the accuracy of a move correctly. Four rules change accuracy. The rules are Coil, Wide Lens, Sand Veil and No Guard. We measured each rule two times. The first measurement had the rule. The second measurement did not have the rule. The two measurements were the same for all four rules. This shows that no rule had an effect.

There are three causes. The first cause is a table that changed the words `accuracy` and `evasion` into an empty value. Eleven parts of the simulator read that table. The second cause is that the simulator did not read items and did not read abilities for accuracy. The third cause is that the function which makes the accuracy decision does not receive the attacker and does not receive the target. A function cannot use an item on a body that it does not have.

The simulator now has one function for accuracy. The name of the function is `hitChance`. The function receives the attacker, the target, the move and the field. Four parts of the simulator call this function. The accuracy decision now happens after the target is known.

The simulator also did not make a Substitute. It removed one quarter of the health of the user. It did not give a Substitute to the user. A player made this move 1,976 times. The move was worse than doing nothing. The simulator now gives a Substitute. A second use of the move fails and does not remove health. One function decides if a move goes through a Substitute. The damage rules and the status rules both use that function.

The count of tested mechanics is 231 of 232. One mechanic is missing. The mechanic has a written reason. Five more rules are known to be absent. Each of the five has a written reason and a usage count. The largest is Aura, which needs information the damage function does not receive.

Change record for 3.50.0. The simulator did not do Taunt. It recorded the Taunt condition. It did not read the condition. A Pokemon with Taunt could still use a status move. The simulator now refuses a status move at two times. The first time is move selection. The second time is move execution. The rule comes from the tag data. It does not use the move name.

A pivot move had the priority of a switch. Parting Shot moved before all other moves. This is not correct. A pivot move is a move. It now uses its own priority.

Volt Switch changed the user when the target absorbed the move. This is not correct. The user now stays if the move does no damage.

Yawn worked against Good as Gold. This is not correct. Good as Gold refuses all status moves from an opponent. The Yawn code now asks.

The count of known differences with the official engine went from 94 to 72.

Change record for 3.49.1. The mutation harness reported 97 defect candidates. The two largest were not defects. The engine reads the Life Orb tag for the damage. The engine uses the item name only for the recoil. This is latent. The engine ignores the Light Screen tag value on purpose. The tag holds the singles value. This is a doubles engine. The doubles value is different. The harness now grades each open operator. It uses four classes. Class A means no lookup and no name branch. Class B means the engine substitutes its own value. Class C means the engine uses the name. Class D means the test could not move the value. The grade comes from a parse of the engine source. It does not come from a comment. No defect candidate is class A. The ratchet counts class A only. Class A is 163 operators. Three known cases test the rule. The harness stops if the rule fails these cases.

Written in ASD-STE100 Simplified Technical English. Sentences are short. The voice is active. One word has one meaning. The document follows the Diátaxis structure: Tutorial, How-to, Reference, Explanation.

1. Tutorial — run ABRA for the first time

Do these steps in order.

1. Get the code. Clone the repository `github.com/willhoop/abra`.
2. Get the sibling engine. Clone `github.com/willhoop/chomp` next to it. ABRA reads it at `../CHOMP`.
3. Open the site. Open `web/index.html` in a web browser. The site needs no build step and no server.
4. Visit a model. Click a house in the town. Open **PORY** and move the sliders. The win % updates.
5. Run one check. In a terminal, run `node engine/validate_damage.js`. It confirms the damage engine.

You now have the site and one validated model.

2. How-to — common tasks

Repair the raw archive before any reparse. `MODE=backfill node engine/durable-ingest.js data/games.ladder.jsonl` The hourly Action appends to the store while the raw archive is gitignored, so CI-collected games have no local raw log. `MODE=reparse` REFUSES to run while any stored game is missing one, because reparse rebuilds the store from the archive and would delete them.

Read which configuration produced a result (added 3.33.0). `node engine/run_stamp.js --show data/rollout-r3.json` Every gate artifact has a `<name>.meta.json` file beside it. The file records the rollout budget, the exploration rate, the horizon, content digests of every source the gate reads, the commit, and whether the working tree was dirty. `node engine/status.js` prints the headline of that file under the gate. Read three fields before you quote a result. `reconstructed: true` means the stamp was inferred from a commit and not observed; read `confidence` beside it. `git.dirty: true` means the commit does not describe what ran; use `source_digests` instead. `source_digests` holds hashes of working-copy bytes and `git.blobs` holds git object names. Do not compare the two. On Windows they differ because git changes the line endings.

Do not quote an engine-speed figure from a document (added 3.62.2). There is no script in this repository that measures the speed of the two engines. Three figures are on record for MEDICHAM — 3,401 battles/sec in ADR-001, 1,606 battles/sec in ROADMAP #61 and 13,041 turns/sec in the 3.62.2 correction. No artifact holds any of them. No test compares them. No ratchet fails when one of them moves. If you need a speed figure, measure it, record the method beside it, and state the unit: `turns/sec` compares the two engines and `battles/sec` does not, because the two engines end a battle under different rules.

Write a stamp from a new measurement (added 3.33.0). Call `require('./run_stamp.js').writeStamp({...})` at the point the run writes its numbers. Do not write a second sidecar format. One artifact recorded a probability column and did not record the exploration rate that produced it. A file written at rate 0 and a file written at rate 1 were then identical byte for byte, and they differed by almost four accuracy points. The published result could not be recovered.

Show the project state on a web page (added 3.32.0). `node web/build-status.js` then open `web/status.html`. The build step writes `web/status-data.js`. The page reads a script-tag global. Do not change it to `fetch()`. A `fetch()` of a local file fails under `file://` and shows no error to the reader.

Show the model select screen (added 3.32.0). Open `web/stadium.html`. Run `node tests/test-stadium-roster.js` after you add or remove a model. The test compares the page against `docs/MODELS.md`. The test fails if a model has no cabinet.

Compare the engine with Showdown (changed 3.32.0). `SHOWDOWN_PATH=... node tests/test-engine-diff.js --seed 20260804` The sampler is seeded. Two runs with the same seed give the same result. Before 3.32.0 the sampler used `Math.random()` and the count changed between runs. Always record the seed with the count.

Check that every room parses (added 3.34.0). `node tests/test-web-parses.js` This test runs the inline script of each page through a parser. A page can contain correct text and still fail to run. On 2026-08-04 a page had one wrong quotation mark. The page showed only its title. Two other tests gave the page full marks, because they read the page as text.

Check that the live site has every room (changed 3.34.0). `node tests/test-site-sync.js` `app/` is the folder the web server uses. `web/` is the folder you edit. The test now compares EVERY page in `web/` against `app/`, and also the data files that a page loads. Before 3.34.0 it compared one file only, so a NEW page that was never copied stayed invisible. If the test fails, run `cp web/<page> app/<page>`.

Check whether an artifact is too old to trust (changed 3.34.0). `node engine/provenance.js` The report gives a drift percentage and, beside it, `ci_gain` and `max_shift`. Use `max_shift`. It states how far the missing games can move the result. A percentage of a store that grows every hour only states the AGE of the artifact. No artifact in this project can move a proportion by one percentage point.

Run the official Champions engine. `SHOWDOWN_PATH=/path/to/pokemon-showdown node engine/champions_sim.js` Needs a BUILT master checkout; the champions mod is not in the npm package.

Generate self-play games (MEW). `SHOWDOWN_PATH=... node engine/mew.js --n 1000` then `SHOWDOWN_PATH=... node engine/validate_selfplay.js` Output goes to `data/games.selfplay.jsonl` and must NEVER be pooled with the ladder store.

Fit the scoring bot's policy. `SHOWDOWN_PATH=... node engine/fit_policy.js` Reads every clean open-team-sheet game, builds one row per human decision, fits by conditional logit, and writes `data/policy-weights.json`. It prints the held-out comparison against the behaviour clone; if the fit does not beat the clone it says so in plain words and the weights must not ship. `engine/selftest.js` refuses a weight file that lost to the clone, and refuses one whose feature list does not match what `engine/board.js` computes.

Check the corpus the policy was fitted on. `SHOWDOWN_PATH=... node engine/corpus_shift.js` Compares open-sheet against closed-sheet ladder on the same code: team composition, behaviour given a board, bot contamination before and after filtering, and mean rating. Run it before trusting a refit. The teams differ by 551.9 points of total absolute species difference; the measured behaviour differs by at most 1.49. `fit_policy.js` then re-estimates on a sample reweighted to the closed-sheet species mix and reports the shift **in standard errors**. Five weights move materially — `priorLogP` by 10.8 SE, `bp` by 6.2 SE (its sign flips) — so the **reweighted vector ships**. Board-reading weights (`eff`, `immune`, `deadStatus`) do not move: reading the board transfers between the two metagames, how much popularity is worth does not.

Pull the Bo3 open-team-sheet ladder. `PAGES=6 CONC=20 FORMATS=gen9championsvgc2026regmbbo3 node engine/durable-ingest.js data/games.bo3.jsonl` CI does this hourly. This format's ruleset carries `Force Open Team Sheets`, so every game publishes all six sets of both sides — the only continuously-collected corpus in which the choice set of a decision is known. The main ladder format carries plain `Open Team Sheets`, which is optional and needs both players to agree, which is why only ~1% of the closed store has sheets. **It goes in its own store and is never pooled with the ladder store** — different information regime, different metagame. Dedupe it with `python3 engine/dedupe_store.py data/games.bo3.jsonl --write`.

Play with the scoring policy. `SHOWDOWN_PATH=... node engine/mew.js --n 1000 --policy score`

--policy	what it does
random	Showdown's <code>RandomPlayerAI</code> . Correct for plumbing and matchup structure; not valid as training data
prior	samples the move a species actually clicks, from <code>data/move-priors.json</code> . Board-blind
score	tracks the board and scores every (move, target) pair with the fitted weights. The only mode that aims

Check the run's own accounting: `policy=score` must report ~100% of decisions scored and a non-zero `aiming:` line. A run reporting 0% is not a scoring bot.

Compare two policies fairly. Pass the SAME `--seed` to both runs — MEW derives its team sampling from it, so the two corpora play identical teams and the comparison is paired rather than confounded by which teams each happened to draw:

```
node engine/mew.js --n 600 --policy prior --seed 4242 --out data/_a.jsonl
node engine/mew.js --n 600 --policy score --seed 4242 --out data/_b.jsonl
node engine/realism_report.js --self data/_a.jsonl
node engine/realism_report.js --self data/_b.jsonl
```

Generate self-play at scale (the farm). `SHOWDOWN_PATH=... node engine/mew_farm.js --n 200000 --procs 12 --conc 1`

`--conc` must stay at 1. The simulator is synchronous and CPU-bound, so in-process concurrency never overlaps real work; it only holds N battles live at once and multiplies GC pressure. Measured on 8 physical / 16 logical cores: 8 procs at `--conc 4` gave 11 games/sec, the same 8 procs at `--conc 1` gave 38. Process count is the unit; 12 procs / conc 1 reproduced at 44–46 games/sec.

Two files are written, and **both are needed**:

file	contains	read by
<code>data/games.selfplay.jsonl</code>	game-level records (six / brought / lead / winner)	store-shaped engines
<code>data/games.selfplay.raw-logs.jsonl</code>	full protocol logs, {id, uploadtime, log}	PORY, <code>state_encoder.py</code>

The value models reconstruct per-turn board states by replaying the **protocol log**; the records are summaries and contain no per-turn state. A corpus written without the sidecar is unreadable by the models it exists to train. Budget ~5 KB per game per file.

Every battle is replayable. A record carries `id`, `selfplay.seed`, `selfplay.policy` and `selfplay.engine_commit`, and both the battle dice and both players' decision PRNGs are derived from that seed. Re-running the same seed against the same pinned engine reproduces the game byte-for-byte (verified 25/25, excluding the `|t:|` wall-clock line). This is what makes a claim like "this switch won the game" checkable rather than asserted.

Teams are validated against Showdown, not against our own rules. `packTeam` enforces Item Clause during packing, then runs the official `TeamValidator` and repairs what it can; MEW discards anything still invalid rather than recording it. `BattleStream` does **not** validate — it plays whatever it is handed — so without this gate an illegal team produces a record indistinguishable from a legitimate game. Before the gate, the validator rejected 80.5% of the pool. Cost is 4.30 ms/team, ~9.6% of a battle.

Team preview is sampled, not fixed. `chooseTeamPreview` draws four of six weighted by measured $P(\text{brought} \mid \text{on team})$ and two leads weighted by $P(\text{lead} \mid \text{brought})$, from `data/bring-priors.json` (regenerate with `node engine/bring_priors.js`). `MEW_PREVIEW_TEMP` controls how far the draw wanders from the common line: 1.0 samples proportional to measured propensity, lower collapses toward the single most likely bring, higher flattens toward uniform.

Refresh the official priors. `node engine/fetch_smogon_stats.js` then `node engine/smogon_priors.js` CI does this on the 4th and 11th of each month.

Pull new replays. `PAGES=6 CONC=20 node engine/durable-ingest.js data/games.ladder.jsonl` The command adds only new games. It never duplicates a game and never re-fetches a stored game.

Rebuild the meta model. `node engine/analyze.js data/games.ladder.jsonl` writes `data/meta-usage.json`.

Refresh the site data. `python3 engine/refresh-site-data.py` writes `data/live.js`, `data/archetypes.json`, and `data/kad-replays.js`.

Run a model or an evaluation.

- GURU matchup matrix: `python3 engine/guru.py`
- XATU belief / policy eval: `python3 engine/eval_policy.py`
- PORY value net: `python3 engine/pory.py`
- CHOMP-EV proof: `node engine/chomp_ev.js`
- SLOWKING preview-Nash (species): `python3 engine/slowking_preview.py`
- SLOWKING preview-Nash (playstyle): first `node engine/playstyle.js` , then `MATRIX_FILE=data/playstyle-matchups.json TAG=playstyle python3 engine/slowking_preview.py`

Validate the damage engine. `node engine/validate_damage.js` . It fails if any scenario is more than 5% from the Smogon damage calculator.

Run the tests. `node tests/test-parse.js` , `node tests/test-dynamics.js` , `node tests/test-medicham.js` , `node tests/test-chomp-ev.js` , `python3 tests/test-jolteon.py` , `python3 tests/test-slowking.py` .

Edit the site, then mirror it. After you change `web/index.html` , copy it: `cp web/index.html app/index.html` .

3. Reference

3.1 Stored game record (`data/games.ladder.jsonl` , one JSON object per line)

Field	Meaning
<code>id</code> , <code>date</code>	replay id and upload time
<code>p1</code> , <code>p2</code>	<code>{name, rating, bot}</code> per player
<code>six.p1/p2</code>	the six revealed at preview
<code>brought.p1/p2</code>	the four actually brought
<code>lead.p1/p2</code>	the two led
<code>sets</code>	per species, the revealed moves / item / ability
<code>turns</code>	per-turn events (move, damage, faint, status, field)
<code>winner</code>	the winning name

3.2 Model outputs

File	Written by	Contents
<code>data/damage-validation.json</code>	<code>validate_damage.js</code>	damage error against the Smogon damage calculator
<code>data/guru-matchups.json</code> , <code>guru.js</code>	<code>guru.py</code>	archetype matchup matrix, Wilson CIs
<code>data/xatu.json</code> , <code>xatu.js</code>	<code>xatu.py</code>	opponent set/move belief
<code>data/pory.js</code> , <code>pory-eval.json</code>	<code>pory.py</code>	mid-game value net + its score
<code>data/chomp-ev.json</code>	<code>chomp_ev.js</code>	the bring proof (null result)
<code>data/playstyle-matchups.json</code>	<code>playstyle.js</code>	playstyle matchup matrix
<code>data/slowking-eval.json</code> , <code>slowking-playstyle-eval.json</code> , <code>slowking*.js</code>	<code>slowking_preview.py</code>	preview equilibrium + exploitability
<code>data/policy-weights.json</code>	<code>fit_policy.js</code>	the scoring bot's fitted weights, the feature list they were fitted against, and the held-out comparison against the behaviour clone
<code>data/meta-usage.json</code> , <code>live.js</code>	<code>analyze.js</code> , <code>refresh-site-data.py</code>	usage model + live site counts

File	Written by	Contents
data/protocol-events.json	derive_protocol_events.js	every event Showdown can emit, which of them <code>medicham2</code> emits, and a written reason for each one it does not

3.2a Protocol trace (`engine/medicham2-browser.js`)

The simulator can emit a Showdown-shaped protocol stream. It is **off by default**; nothing in the battle loop pays for it unless a caller asks.

To turn it on:

```
const trace = [];  
const S = M.battleInit(teamA, teamB, { trace });  
M.battleTurn(S, rng);  
// trace is an array of protocol lines:  
//   |move|p1a: incineroar|fakeout|p2a: garchomp  
//   |-damage|p2a: garchomp|154/175  
//   |cant|p2a: garchomp|flinch
```

To read it:

Export	Does
<code>M.TRACE_EVENTS</code>	the 36 event names this engine claims it can emit
<code>M.traceCounts(lines)</code>	counts by event name, PARSED from the lines rather than kept beside them
<code>M.traceCanon(line)</code>	the one normaliser — lowercase and strip whitespace per field

Identifiers are ids, not display names. The engine writes `p1a: incineroar` and `fakeout` where Showdown writes `p1a: Incineroar` and `Fake Out`. This engine holds no display-name table and inventing one would be a translation layer that can itself be wrong. `traceCanon()` is applied to **both** streams by any comparison driver, which makes the two agree by canonicalisation rather than by translation.

Do not add an event without regenerating the artifact. `node engine/derive_protocol_events.js --write` fails if a name in `TRACE_EVENTS` is one Showdown never emits, and fails if a Showdown event is neither emitted nor given a reason. `tests/test-protocol-trace.js` fails if a claimed event never fires in a real game.

3.3 Continuous collection

A GitHub Action (`.github/workflows/ingest.yml`) runs the pull hourly and commits the store. A separate tests workflow runs the test suite and the damage validation on every push and pull request.

4. Explanation

4.-2 Why the exploitability is the headline metric (added 3.62.2)

This section explains a decision. It does not give instructions. For the decision itself, read ADR-003.

There are two kinds of game. In the first kind, both players see all of the state. Chess and Go are of this kind. A search over that state is correct, and one best move exists at each position. In the second kind, each player holds private information. Poker is of this kind. VGC is also of this kind: you do not see which four of the six the opponent brings, the items, the abilities, or the fourth move of a set.

In the second kind of game, one best move does not exist. The correct object is a mixed strategy. If you always make the same choice in the same situation, an opponent who watches you can find that choice and defeat it. This is why the poker research of 2007 to 2021 produced CFR, DeepStack, Libratus and ReBeL,

and not a larger chess search.

A fixed policy is therefore exploitable by construction. A behaviour clone is a fixed policy. A PPO agent is a fixed policy. Both give one answer for one state. VGC-Bench measured this on its own agents: it trained a best response against each agent and found approximately 100% exploitability, although one of those agents defeats a professional player. The two facts are consistent. Strength on average and readability under study are different quantities.

A win rate cannot show this defect. A win rate is measured against a population that does not adapt. An exploitability is measured against an opponent that is trained against you. Only the second measurement finds a policy that is strong today and readable after five games.

The claim under test in this project. A search that computes a new answer each turn shows no fixed map to an opponent. It should therefore be more difficult to exploit than a compiled policy. Three properties of VGC can defeat this claim, and all three are open:

property	why it can defeat the claim
simultaneous moves	sequential CFR does not apply directly to a turn node
stochastic resolution	damage rolls, accuracy and speed ties add variance to every leaf
short horizon	a median game is 6 turns, so there is little depth for a search to use

Do not read this section as a result. The project has no exploitability figure. `data/exploitability.json` is declared void. The comparison with approximately 100% is prepared and not yet made.

4.-1 Why additivity is the recurring failure (added 3.28.0)

The same defect has now appeared at three levels of this project, and it is worth stating once because the fix has the same shape every time.

A sum of independent terms cannot express a conjunction. If a score is $w1*a + w2*b$, there is no setting of `w1` and `w2` that makes "a AND b together" worth more than the parts. Not *scored badly* — **not representable**.

Where it bites:

level	the thing that cannot be said	consequence
one Pokémon (PORY, §4.0)	"material lead matters on turn 3, not turn 25"	the neural net exists for this
two Pokémon (MAG → DODUO)	"Protect with A while B removes the thing killing A"	independent per-slot choice; MAG aims both attacks at one foe ~50% of the time where humans do 23.4%
six Pokémon (JOLTEON → DITTO)	"Pelipper + Archaludon is worth more than the sum"	measured: the additive species block moves the score by ~6.3 while the two non-additive terms move it by ~0.4, so hill-climbing converges on the top-6 by weight

The literature agrees and names the boundary. Cooperative multi-agent value factorisation (QMIX, Rashid et al. 2018) constrains the joint value to be *monotonic* in each agent's utility, and QPLEX and Weighted QMIX exist precisely because that constraint cannot represent non-monotonic coordination. Our case is the non-monotonic one.

But the expensive machinery does not apply here. That literature is about avoiding enumeration when agents are many. With **two** Pokémon the coordination graph is a single edge, so Variable Elimination degenerates to enumerating the joint actions and Max-Plus message passing is unnecessary. Measured on a real mid-game board: **9 × 8 = 72** joint actions per side, of which only **28** have a non-zero interaction term — the other 44 score exactly as the sum of two singles already computed.

And the fix is not "add the interaction term", it is "fit it for the right thing." DODUO's pair block exists and is fitted; it loses at 42.0% because it was fitted to *resemble human pairs* rather than to *win*. Expressiveness was necessary and not sufficient.

Practical rule. Before adding a feature, ask whether the thing you want to say is a conjunction. If it is, no weight on an individual term will ever say it — and if the model already has the interaction term, check what objective it was fitted to before concluding the idea failed.

4.0 What a neural network is here, and why it is not automatically an upgrade

Every model in ABRA answers one question: given the board right now, what is $P(I \text{ win})$? PORY answers it with **logistic regression** — multiply each feature by a weight, add them up, squash to 0..1:

$p = \text{sigmoid}(w_0 + w_1 \text{alive_diff} + w_2 \text{hp_diff} + \dots)$

That can only draw a straight dividing line through feature space. It cannot express "*being one Pokémon ahead matters enormously on turn 3 and barely on turn 25*", because that is an **interaction** between two features, and a sum of independent terms has no way to say it.

A **neural network** is the same idea with a middle step. Inputs are combined into H hidden units, each its own weighted sum passed through a nonlinearity (ReLU: $\max(0, x)$), and those are combined into the answer:

$h = \text{relu}(W_1 @ x + b_1)$ # H learned intermediate quantities $p = \text{sigmoid}(W_2 @ h + b_2)$

Each hidden unit can become a detector for a **conjunction** — "ahead on material AND my active is faster AND Trick Room is not up" — and the output layer weighs the detectors. Universal approximation (Cybenko 1989; Hornik 1991) says one hidden layer of sufficient width can represent any continuous function on a bounded domain, so the network is strictly **more expressive** than the linear model.

Strictly more expressive is a claim about representation, not about learning. Extra capacity spent on features that contain no interactions buys nothing and costs variance. `engine/pory_baseline.py` already established the relevant fact: PORY's six material features are **beaten by two of them** (`alive_diff + hp_diff` at 0.5822 vs PORY's 0.5840). If the features carry no more signal, a network fitted to them lands in the same place — and reporting otherwise would be measuring the estimator rather than the game.

This is the lesson the game-playing literature learned repeatedly. **TD-Gammon** (Tesauro 1994) needed hand-designed board features, not checker counts. **AlphaGo Zero / AlphaZero** (Silver et al. 2017, 2018) feed the value head a *stack of planes* — piece positions, repetition, side to move — because material is precisely the baseline a value net must beat, and it beats it by seeing **where** the pieces are. Leela Chess Zero's ablations show the value head collapsing toward the handcrafted evaluation when the positional planes are stripped.

So the binding constraint was **representation, not capacity**. `engine/state_encoder.py` supplies the planes: HP per slot, active vs benched, status, stat boosts, weather / terrain / Trick Room / Tailwind / screens, hazards, and the active Pokémon's types — 121 features where PORY had 6.

`engine/pory_nn.py` then separates the two explanations rather than conflating them, by running eight arms on one split:

arm	what it isolates
B2 logistic, <code>alive_diff + hp_diff</code>	the bar — two material features
L6 logistic, PORY's six	PORY itself
LR logistic, rich features	gain from representation alone
N6 network, PORY's six	gain from nonlinearity alone
NR network, rich features	both

If the network wins only on `N6`, the gain is nonlinearity. If only on `LR`, PORY was feature-starved rather than model-starved. If neither beats `B2`, that is the result and it is reported as such.

Species identity is deliberately **excluded**: ~240 species one-hot across 8 slots is a 1,920-dimensional input, which on ~9,000 real games would be fitted almost entirely to noise. Species enters only through type. Embeddings become defensible once the self-play corpus is large enough, and the encoder is versioned so that change is visible rather than silent.

Two methodological points that decide whether any of these numbers mean anything. **Splits are by game, never by state** — turns within a game share an outcome, so splitting on states leaks the label across the boundary and flatters every arm equally. And **every state is emitted twice with the sides swapped**, because antisymmetry in the two players is a property of the game; a model trained on p1's view alone will not respect it.

Store raw, analyse on top. ABRA saves every game with every fact it may ever need. Every filter — rating tier, humans-only, archetype, playstyle — runs over the store at read time. A change to how the games are segmented is a re-computation, not a re-download. This makes the fetch a one-time cost and keeps the analysis free to change.

Support decisions, do not predict outcomes. In this format, the winner of a game is near-impossible to predict from the two team sheets; even a player-rating model ties a coin. ABRA therefore judges each model on a decision, not on the match result. Each probability ships a proper score (log-loss or Brier), a confidence interval, and an honest baseline.

Why the confidence interval is clustered. States within one game are correlated. A confidence interval that resamples states would be too narrow. ABRA resamples whole games instead. For a matchup matrix, it also resamples each cell from its Beta distribution before it solves, so the interval carries the small-sample uncertainty.

Honest negatives are kept. Two results are negative and are reported plainly: the team-picker's brings do not beat a coin (CHOMP-EV), and the playstyle rock-paper-scissors cycle rests on small samples and is suggestive, not settled. A negative that is measured is more useful than a positive that is asserted.

Companion documents. [White paper](#) · [Deck](#) · [Project summary](#) · [Model ledger](#) · [Changelog](#)

How to measure against a frozen engine

Description

An engine release is a copy of every file whose content can change a measured number. A measurement reads the copy. Other work on the repository does not change the copy.

The set of frozen files is declared as `SOURCES` in `engine/engine_release.js`. Read it from there.

```
node -e "console.log(require('./engine/engine_release.js').SOURCES.join('\n'))"
```

Do not copy that list into this page. It has grown four times — loader dependencies were added so `REL.require` can resolve, lazily-read data files were added so a snapshot can actually play a game, and the fitted weights were added because "can anything beat MAG" is a claim about one specific vector. This paragraph said **twelve** and named twelve files until 2026-08-22, by which point the declaration held more than twice that many. A release that is a valid digest set but not a loadable engine is the failure the additions each fixed, and a typed list here cannot track them.

The release identifier is a digest of the file digests. If the files do not change, the identifier does not change. A second cut of the same files makes no second copy.

Procedure — cut a release

1. Make sure the files you want to freeze are correct.
2. Run this command. Give a reason.

```
node engine/engine_release.js cut "why this release exists"
```

1. Read the digests in the output — one per entry in `SOURCES`, and the count is whatever that declaration currently holds. Confirm the Showdown commit is not `UNKNOWN`.

Procedure — measure against a release

1. Open the release at the start of the measurement.
2. Load every engine file from the release, not from `engine/`.
3. Put the stamp in the artifact you write.

```
const REL = require('./engine_release.js').open();
const MEDI = REL.require('engine/medicham2-browser.js');
const artifact = Object.assign({}, REL.stamp(), result);
```

`REL.stamp()` writes `engine_release`, `showdown_commit` and `source_digests` into the artifact. `engine/provenance.js` reads `source_digests` and compares the content. An artifact without this stamp can only be checked by timestamp, and a timestamp cannot show that a file changed after it was read.

Procedure — check a release

```
node engine/engine_release.js list           # each release, and how many files have moved since
node engine/engine_release.js verify <id>    # has the copy itself changed?
```

`list` shows the drift. Drift is normal. Drift tells you if an old number still describes the engine that ships today.

Warning

Do not read `engine/` files during a measurement. Another division can write them at any time. This does not cause an error. It causes a number that is wrong and looks correct.

On 2026-08-04 three divisions ran at the same time with separate files. The weights of the model under test changed between the two halves of one measurement. The measurement completed. No check failed. 7,100 games were lost.